tmux documentation

Per-Session Tmux Isolation — Plan

Source: Web deep-research dispatched 2026-05-07. See repository CONTINUATION.md §3.2 for status.

Recommended pattern: `systemd-run --user --scope`

Each `tmx new <name>` creates a transient systemd `--user --scope` cgroup-v2 unit named `tmx-<name>.scope` with:

- `MemoryMax=8G` (default; tunable via `TMX_MEM` env var)
- `CPUQuota=200%` (= 2 CPUs; tunable via `TMX_CPU`)
- `Delegate=yes` (lets tmux/children create sub-cgroups for LSPs etc.)
- `TasksMax=4096`

A separate tmux server runs INSIDE each scope (own `-L <name> -S /run/.../<name>.sock` socket). Crash isolation: when the scope hits its memory cap or a fatal signal, **only that scope dies**. Other scopes and the user session are unaffected (same property §12.7 already validates for AOSP builds).

Why NOT podman-per-session

Concern	systemd scope	podman container
FS access (`$HOME`, ssh keys, `~/.tmux.conf`)	**Native, zero config**	Needs explicit bind-mounts
Network (host VPN, DNS)	**Native**	slirp4netns slow, port <1024 blocked
TTY (xterm-256color)	**Native PTY**	`podman exec -it` has OPOST/ONLCR glitch (containers/podman#3179)
Per-keystroke latency	**Zero**	+1 PTY copy (conmon)
Setup complexity	~40 line bash wrapper	image build + volumes + userns mapping
Nesting (parent already containerized)	**Fine** — scopes nest in user.slice	Pidfd/userns drama
Host deps	systemd ≥ 230, cgroup v2	podman/docker + conmon + slirp4netns

systemd scope wins on every dimension that matters for an interactive shell tmux session.

Wrapper API (subcommands)

```
tmx new <name>          create + attach (one server per scope)
tmx attach <name>       reattach
tmx ls                  list running scopes with mem/cpu usage
tmx kill <name>         systemctl --user stop tmx-<name>.scope + cleanup s
```

Tunables (env vars):

- `TMX_MEM` (default `8G`) — `MemoryMax` per scope
- `TMX_CPU` (default `200%`) — `CPUQuota` per scope

Crash isolation invariants (must be tested)

Invariant	Test
OOM-kill confined to one scope	`tmx new victim` → exhaust 16G in victim → only `tmx-victim.scope` exits, `tmx-survivor` alive
User session survives	`loginctl is-active user@<uid>.service` stays `active` throughout T1+T2
CPU quota enforced	spawn 16 hot threads → `systemd-cgtop` shows ≤ 220% sustained
SIGKILL containment	`kill -9 -- -$$` inside scope → only that scope dies
Concurrent sessions	5 sessions × Neovim + LSPs → all 5 visible, sum < 5×cap
Detach/attach idempotent	`$$` unchanged across detach/attach round-trip
Mutation of `MemoryMax=` line FAILS T1	Anti-bluff §1 covenant proof

See `scripts/tests/` for actual test scripts (to be written in v2 implementation phase).

Edge cases & gotchas

1. `KillUserProcesses=yes` (systemd ≥ 230 default): logout would kill scopes. Run `loginctl enable-linger $USER` to survive logout.
2. **One tmux server per scope** = no cross-session window-move (acceptable trade-off — single-server defeats isolation).
3. `Delegate=yes` **mandatory** for child cgroups (LSPs, build watchers).
4. **Cgroup v1 hosts**: limited support. Add `systemd.unified_cgroup_hierarchy=1` to kernel cmdline on Ubuntu 20.04 / RHEL 8.
5. **RHEL 8** has known systemd bug — MemoryMax not enforced in transient `--user` scope. RHEL 9 fine.
6. `PrivateTmp=yes` would break ssh-agent / gpg-agent sockets — DO NOT enable on the scope.

Recommended RAM cap rationale

Heavy interactive tmux session (Neovim + LSPs + treesitter + htop + build watcher + 2-3 MCP servers) typically holds 2-4 GB RSS, with rare spikes to 6 GB during heavy compile/dexopt. Default `8G` gives 2× headroom. On a 64 GB host, 5 concurrent sessions × 8 GB cap = 40 GB — fits below 60% (38 GB) cap if not all run heavy at once. Realistic concurrent heavy-use is 2-3 sessions.

Sources

- Ankur Sinha — Isolating tmux windows — primary prior art
- tmux/tmux#428 — confirms no upstream feature
- systemd.resource-control(5) — semantics

- containers/podman#3179 — pattern-A TTY cost
- Red Hat KB 7099227 — RHEL 8 MemoryMax bug